

Zadanie indywidualne: Fabryka Abstrakcyjna + Fluent API w C#

Temat: System tworzenia wypraw w świecie fantasy

Czas wykonania: około 30–45 minut

Poziom: średni

Cel: przećwiczenie dwóch wzorców/prowadzeń projektowych naraz: **Abstract Factory** oraz **Fluent API**

1. Opis sytuacji

W świecie fantasy różne rody przygotowują wyprawy wojenne, dyplomatyczne lub zwiadowcze.

Każdy ród ma własny styl działania i własne typy jednostek. Na przykład:

Ród / frakcja	Wojownik	Dowódca	Wsparcie
Starkowie	Strażnik Północy	Lord Dowódca	Wilkor
Lannisterowie	Rycerz Lannisterów	Strateg Dworu	Złoto
Targaryenowie	Gwardzista Smoka	Matka Smoków	Smok

Twoim zadaniem jest napisanie aplikacji konsolowej w C#, która pozwoli tworzyć wyprawy dla różnych rodów.

Program ma korzystać z:

1. **Fabryki Abstrakcyjnej**, aby tworzyć zestaw spójnych obiektów dla danego rodu.
2. **Fluent API**, aby wygodnie i czytelnie konfigurować wyprawę krok po kroku.

2. Cel zadania

Zaprojektuj program, w którym można utworzyć wyprawę w stylu:

```
var mission = new MissionBuilder()
    .ForHouse(starkFactory)
    .WithGoal("Obrona Północy")
    .AtLocation("Winterfell")
    .WithRiskLevel("wysokie")
    .Build();
```

To tylko przykład stylu wywołania. Kursant może zaprojektować własne nazwy metod.

Najważniejsze jest to, aby kod był czytelny i pokazywał połączenie:

- fabryki tworzącej rodzinę obiektów,
- płynnego budowania konfiguracji wyprawy.

3. Wymagania funkcjonalne

Program powinien umożliwiać utworzenie przynajmniej **dwóch różnych wypraw** dla dwóch różnych rodów.

Każda wyprawa powinna zawierać:

- nazwę rodu lub frakcji,
- cel wyprawy,
- miejsce wyprawy,
- wojownika,
- dowódcę,
- element wsparcia,
- poziom ryzyka.

Na końcu program powinien wypisać opis każdej wyprawy w konsoli.

4. Wymagania dotyczące Fabryki Abstrakcyjnej

Utwórz abstrakcyjny mechanizm tworzenia rodziny obiektów.

Przykładowe elementy do utworzenia:

- IWarrior
- ICommander
- ISupport
- IHouseFactory

Fabryka konkretnego rodu powinna tworzyć spójne obiekty dla tego rodu.

Przykładowe fabryki:

- StarkFactory
- LannisterFactory
- TargaryenFactory

Każda fabryka powinna zwracać własne wersje wojownika, dowódcy i wsparcia.

Przykład logiczny:

- StarkFactory tworzy Strażnika Północy, Lorda Dowódcę i Wilkora.
 - LannisterFactory tworzy Rycerza Lannisterów, Stratega Dworu i Złoto.
 - TargaryenFactory tworzy Gwardzistę Smoka, Matkę Smoków i Smoka.
-

5. Wymagania dotyczące Fluent API

Utwórz klasę odpowiedzialną za budowanie wyprawy w sposób płynny.

Przykładowe metody:

- ForHouse(...)
- WithGoal(...)
- AtLocation(...)
- WithRiskLevel(...)
- Build()

Metody powinny zwracać obiekt budujący, aby można było łączyć wywołania w łańcuch.

Przykład oczekiwanego stylu:

```
new MissionBuilder()  
  .ForHouse(...)  
  .WithGoal(...)  
  .AtLocation(...)  
  .WithRiskLevel(...)  
  .Build();
```

6. Oczekiwany wynik działania programu

Program powinien wypisać w konsoli coś w tym stylu:

WYPRAWA: Obrona Północy
Ród: Starkowie
Miejsce: Winterfell
Wojownik: Strażnik Północy
Dowódca: Lord Dowódca
Wsparcie: Wilkor
Ryzyko: wysokie

WYPRAWA: Zdobycie wpływów
Ród: Lannisterowie
Miejsce: Królewska Przysań
Wojownik: Rycerz Lannisterów
Dowódca: Strateg Dworu
Wsparcie: Złoto
Ryzyko: średnie

7. Ograniczenia

Nie twórz jednego dużego konstruktora z wieloma parametrami.

Nie twórz wszystkich obiektów ręcznie w Main.

Nie używaj dużej liczby instrukcji if lub switch do wybierania typu wojownika, dowódcy i wsparcia.

Dobór obiektów dla danego rodu powinien wynikać z zastosowania **Fabryki Abstrakcyjnej**.

Konfiguracja wyprawy powinna być czytelna dzięki **Fluent API**.

8. Co należy oddać?

Kursant powinien oddać:

1. Projekt konsolowy C#.
2. Kod źródłowy z użyciem Fabryki Abstrakcyjnej.
3. Kod źródłowy z użyciem Fluent API.
4. Minimum dwie utworzone wyprawy.
5. Wynik działania programu w konsoli.
6. Krótką odpowiedź na pytania:

Dlaczego w tym zadaniu pasuje Fabryka Abstrakcyjna?

Dlaczego w tym zadaniu przydaje się Fluent API?

9. Pytania kontrolne

Po wykonaniu zadania odpowiedz krótko:

1. Co tworzy Fabryka Abstrakcyjna w Twoim programie?
 2. Dlaczego konkretna fabryka nie powinna tworzyć obiektów innego rodu?
 3. Która część programu odpowiada za płynne API?
 4. Dlaczego zapis łańcuchowy jest czytelniejszy niż wiele osobnych przypisań?
 5. Co trzeba zmienić, aby dodać nowy ród, np. Greyjoyów?
-

10. Wariant dodatkowy dla szybszych osób

Dodaj trzecią frakcję, np.:

Frakcja	Wojownik	Dowódca	Wsparcie
Greyjoyowie	Korsarz Żelaznych Wysp	Kapitan Floty	Okręt
Nocna Straż	Zwiadowca	Lord Komandor	Kruki
Dzicy	Wojownik Wolnych Ludzi	Przywódca Klanu	Olbrzym

Następnie utwórz trzecią wyprawę i wypisz ją w konsoli.

Sens zadania

W tym ćwiczeniu **Fabryka Abstrakcyjna** odpowiada za stworzenie spójnej rodziny obiektów dla danego rodu, a **Fluent API** pozwala wygodnie skonfigurować wyprawę w czytelnej, niemal zdaniowej formie.

Fabryka daje porządek. Fluent API daje elegancję. Razem robią mały projekt, który nie wygląda jak skład części z rozbitego wozu, tylko jak zbrojownia przygotowana do wymarszu.